Decoupled Information-Set MCTS with a Card-Value Model for Sushi Go!

Halit Ozger
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 2]
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 3]
Student no. [to be completed]
Queen Mary University of London
London, UK

***Abstract*—In the card-drafting game Sushi Go! two to five players pick cards simultaneously from hidden hands that are passed around the table, so a planning agent faces hidden information, simultaneous moves and delayed scoring. Starting from TAG's BasicMCTS, we added a model of the game's scoring that drives both the evaluation and the rollouts, rollouts that end with the round, max-n backups, decoupled UCT for the simultaneous choices and single-observer information-set search. In 500 three-player games at 100 ms per decision the agent wins 55.0% [50.6, 59.3] against TAG's tuned ISMCTS and our previous agent (chance 33.3%) and 84–87% against BasicMCTS, RHEA and OSLA. Paired ablations show that the heuristic and the game-matched tree matter most, whereas a much stronger rollout policy does not help.**

Keywords*—*Monte Carlo tree search, information sets, simultaneous moves, card games, Sushi Go!, TAG

I. Introduction

Tabletop games are a long-standing test bed for game AI: they combine chance, hidden information and several interacting players within small, well-defined rule sets. The Tabletop Games framework (TAG) [1] gives such games a common Java interface so that agents can be compared on them. One of its games is *Sushi Go!* (P. Walker-Harding, Gamewright, 2013): every player secretly picks one card from its hand, all players reveal their cards at the same time and pass the rest of their hands to the next player, and after three rounds the player with the most points wins. Klang et al. [2] introduced the game to TAG and assessed simultaneous action selection and complete information in it; the current version, which we use, shows an agent only the cards it has seen.

Four properties make Sushi Go! hard for a search agent. (i) The opponents' hands are hidden until they are passed on, so the true state is unknown at the start of every round. (ii) Moves are simultaneous: no card can be chosen in reaction to an opponent's card of the same turn. (iii) Most points are delayed and conditional – sets pay out only when completed, Maki rolls at the end of the round and Pudding at the end of the game – so short simulations miss them and long ones are noisy. (iv) With up to ten cards per hand and three or four players a turn has hundreds of joint moves, and an agent has 0.1–1 s per decision. TAG's BasicMCTS addresses none of the first three: it searches the single shuffled sample of the hidden cards it receives as if it were the true state, treats the players of a turn as moving one after another and evaluates random rollouts with the raw score.

We ask which adaptations of Monte Carlo tree search (MCTS) to these properties make an agent stronger, and how their value changes with the time per decision and the number of players. We answer this by extending BasicMCTS in steps, keeping every modification as a switch, and measuring each in experiments of 500 games. Our contributions are:

- a model of Sushi Go!'s scoring that estimates every player's final score and every card's marginal value, used both to evaluate states and as the rollout policy of all players;
- a search matched to the game – rollouts that end with the round, max-n backups, decoupled UCT and single-observer information-set MCTS – built on BasicMCTS, which keeps its strength at long budgets, unlike our sequential-tree agent, whose win rate fell from 50% to 33% between 100 and 900 ms;
- settings and a fast simulator for four players;
- an experimental study of 161 runs (67 500 games) with leave-one-out ablations, a 2^4 factorial design, validation on unseen deals and negative results showing that the model of the game, not the search effort, now limits the agent.

II. Background

A. TAG and Sushi Go!

TAG [1] splits a game into a state, a forward model and parameters; an agent receives a copy of the state, in which every card it has not seen has been reshuffled, and uses the forward model to simulate moves. In TAG's Sushi Go! the 108-card deck holds Tempura (5 points per pair), Sashimi (10 per triple), Dumplings (1, 3, 6, 10 or 15 points for one to five), Salmon, Squid and Egg Nigiri (2, 3 and 1 points, tripled on an earlier Wasabi), Maki rolls with one to three icons (6 and 3 points for the most and second-most icons of a round), Chopsticks (two cards in a later turn) and Pudding (+6 for the most and, with three or more players, −6 for the fewest at the end of the game). Each round deals $12 - n$ cards to each of n players. As the hands rotate, every hand of a round has been seen after $n - 1$ passes, so hidden information is concentrated at the start of each round, while the value of a card depends on what the others will take from the hands we pass on.

B. Monte Carlo Tree Search

MCTS [3] builds a search tree by repeating four steps: selection, expansion, simulation (rollout) and backpropagation. UCT [4] selects children with the UCB1 rule [5], maximising $Q + K\sqrt{(\ln N / n)}$ for a child with mean reward Q visited n of N times, which balances exploitation and exploration. Domain knowledge can enter through heuristic rollout policies [6] or through priors such as progressive bias [7]. A stronger rollout policy does not guarantee better estimates, however: Silver and Tesauro [8] showed that rollout outcomes need to be unbiased rather than produced by a strong player (simulation balancing).

C. Hidden Information, Simultaneous Moves and Several Players

Determinization, or perfect-information Monte Carlo (PIMC) sampling, samples the hidden information, searches each sample as if it were the true state and combines the results. Long et al. [9] related its success in card games to properties such as how quickly hidden information is revealed during play, which in Sushi Go! happens within $n - 1$ turns. Its known flaw is strategy fusion: the search assumes that it can act differently in states it cannot tell apart [10], and in a drafting game it plans around opponent cards it cannot know. Information-set MCTS (ISMCTS) [11] avoids this by searching one tree over the searching player's information sets and sampling the hidden information anew in every iteration; it was developed for card games such as Dou Di Zhu. Its single-observer form treats the opponents as if they knew what the searcher knows, whereas the multiple-observer form models their knowledge with one tree per player at a higher cost [11]. In Sushi Go! every player knows its own hand at the start of a round and all hands after $n - 1$ passes, so the single-observer approximation errs only early in each round.

Simultaneous moves are usually handled by decoupled UCT (DUCT): every player chooses its own action at a node from its own bandit statistics and the joint action leads to a child [12]. With UCB selection DUCT is not guaranteed to converge to an equilibrium, so variants with EXP3 or regret matching were proposed [12], and which variant works best depends on the game [13]; these studies used two-player games such as Goofspiel. In multi-player games the max-n backup, in which every player maximises its own value, generally outperforms the paranoid assumption that all opponents play against us [14].

BasicMCTS, the starting point allowed for this work, uses none of these ideas, and the earlier Sushi Go! study in TAG [2] predates the version of the game with hidden hands. ISMCTS and DUCT have been studied separately and mostly in two-player games; how they combine with domain knowledge in a multi-player drafting game, what each contributes, and how this depends on the time budget and the number of players is not known – the questions this report answers.

III. Method

Our agent, SushiMCTS, is a modified copy of TAG's BasicMCTS (package *players.basicMCTS*) and uses no code of TAG's advanced MCTS agent. Every modification is a switch in its parameter class, so all variants in our experiments run the same code. Table I lists the modifications and Fig. 1 shows one iteration of the final agent. Beyond the published techniques we combine, three elements are our own: one card-value model serves both the evaluation and every player's simulation, decoupled selection runs on an open-loop information-set tree whose statistics are keyed by card type, and the agent switches its settings with the number of players.

TABLE I. Modifications of BasicMCTS (✓ = used in the final agent with 3 / 4 players)

Id	Modification	3 p / 4 p
H	heuristic (2)–(3) from the card-value model (1)	✓ / ✓
B	ε-greedy card-value rollouts for every player [6]	✓ / ✓
R	rollouts end when the round is scored	✓ / ✓
P	one branch per distinct card	✓ / ✓
M	max-n backups [14]	✓ / ✓
S	decoupled UCT, one node per turn [12]	✓ / ✓
I	single-observer information-set search [11]	✓ / ✓
X0	card-model constants tuned for 4 players	– / ✓
W2, F, L	win-gap value, fast simulator, 8 rollouts per leaf	– / ✓ [a]
A	progressive bias [7]	dropped in v3
D	ensemble of up to 4 determinized trees [9]	replaced by I
N	denial in the rollouts (Section IV-E)	not adopted

[a] at 500 ms or more per decision.

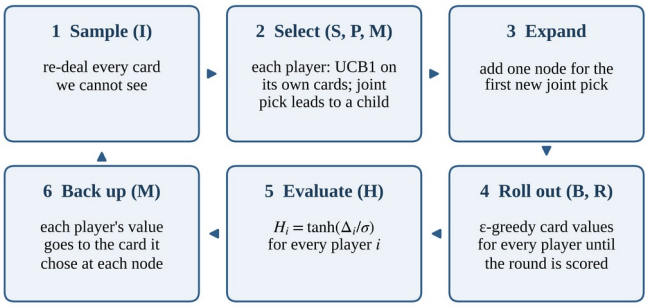


Fig. 1. One iteration of the final agent. The letters refer to Table I; after the time budget the agent plays its most visited card at the root.

A. Card-Value Model and Heuristic (H)

For a state s and player p the model estimates the final score

$$\hat{S}_p = S_p + \alpha U_p + M_p + w_t P_p, \quad (1)$$

where S_p is the banked score (TAG scores Tempura, Sashimi, Dumplings and Nigiri when they are revealed), U_p the value of unfinished sets, each weighted by a completion factor $\min(1, (r - m + 1)/T)$ (0 if $r < m$) for a set that needs m more cards with r picks left, M_p the Maki points p would get if the round ended now, and P_p the Pudding points if the game ended now, weighted by $w_t = t/3$ in round t ($\alpha = 0.5$, $T = 4$). The value of s for player i is

$$H_i(s) = \tanh(\Delta_i / \sigma), \quad (2)$$

$$\Delta_i = w(\hat{S}_i - \max_{j \neq i} \hat{S}_j) + (1 - w)(\hat{S}_i - \mathrm{mean}_{j \neq i} \hat{S}_j), \quad (3)$$

with $\sigma = 15$ and $w = 0.7$: the first term rewards beating the leader, since only first place wins, and the second keeps a gradient when the agent is far ahead or behind. The marginal value $mv_p(c)$ of a card c is the change of $\hat{S}_p$ when p plays it.

B. Simulation (B, R)

B: every player plays its card with the highest *mv*, or a random card with probability $\varepsilon = 0.2$; the same policy is our model of the opponents. **R:** a rollout stops when the current round has been scored. After that only Pudding carries over, which (1) captures, whereas simulating the later rounds would add variance.

C. Tree (P, M, S, I)

P: identical cards lead to identical states, so a node has one branch per distinct card. **M:** a node stores one value per player and every player maximises its own value [14]. **S:** one node represents a whole turn. At a node every player p chooses a card

$$a_p = \text{argmax}_a \left[Q_p(a) + K\sqrt{(\ln N / n_p(a))} \right] \quad (4)$$

from its own statistics, trying untried cards first; the joint choice is played in the forward model and leads to the child of that joint action [12]. In our previous agent (v3) the players of a turn appeared one after another in the tree, so a later player "saw" the card an earlier player had just chosen, which is impossible in the real game. **I:** every iteration starts from a new sample of the cards we cannot see and walks one shared open-loop tree whose nodes store statistics but no states [11]. Statistics are keyed by card type, and $n_p(a)$ counts the iterations in which p held card a. I replaces the determinization ensemble D of v2 and v3, which split the budget between up to four independent trees. With three players K = 0.5; after the budget the agent plays its most visited root card.

D. Four Players

With four players the best settings differ (Section IV-D), so the agent switches parameters by table size. **X0:** card-model constants (Sashimi discount, Maki standing weight, Dumpling, Pudding and Nigiri values, counting the cards still in the hands) found by a search-free coordinate tuner that plays greedy card-value policies against each other. At 500 ms or more the 4-player agent also uses **W2:** w = 1, finished games valued by $\tanh((S_i - \max_j S_j)/2)$ and σ shrinking with the rounds left; **F:** TAG's Sushi Go! rules re-implemented on integer arrays and checked move by move against the forward model, which runs 7–11 times more iterations, spent on **L** = 8 rollouts per leaf; and K = 1.

E. Experimental Protocol

The opponents are our previous agent v2 (H with progressive bias, B and D; K = 1.41) and TAG's tuned ISMCTS (*players.mcts* with information sets, one tree per player, UCB-Tuned, K = 1, MAST and random rollouts), used only as an opponent; further runs use BasicMCTS (K = 1.41, 10-move random rollouts), RHEA (horizon 10, population 10) and OSLA with TAG's score heuristic. All agents get the same CPU-time budget per decision: 100 ms for development and 500 and 900 ms for validation (the competition allows 1 s). In game g of a run our agent sits in seat g mod n, the other seats alternate between the opponents, and the game uses seed $s + g$. Since the v4 study the cards are dealt from a fixed deck order shuffled with this seed, so runs with the same seed play identical deals and seats and are compared game by game; earlier runs (all v3 results and Fig. 2) used TAG's own deal, which differs between runs, and are compared as independent samples. Development used seeds 1 and 2 (and 1001–3001 in v3); the final agent was validated on seed 7, which no decision used. Each condition has 500 games, except 250 at 900 ms and in some 4-player 500 ms runs, where one game costs 48–73 s of CPU time (about 100 runs on an 18-core laptop, 8 games in parallel).

Each game is an independent trial whose outcome for our agent is 1 for a win, 1/k for a k-way shared first place and 0 otherwise. Win rates are reported with Wilson 95% intervals [15], the score margin (our score minus the best opponent's) as mean ± 1.96 standard errors (SE), and differences between runs on the same deals as the mean of the per-game differences ± 1.96 SE, with z = mean/SE. With 500 games a win rate near 50% is known to ±4.4 points; as the outcomes of two agents on the same deal correlate by only 0.1–0.2, a paired difference has a confidence interval of about ±5.7 points. The score margin is about twice as sensitive and served for screening. We expected (H1) every component to improve both measures, (H2) the game-matched tree to matter more at longer budgets and (H3) the best settings to depend on the number of players.

IV. Results

A. From v2 to v4

Fig. 2 traces the development with three players. Removing progressive bias from v2 and adding P, R, M and K = 0.5 doubled the win rate at 100 ms (24.8% → 50.1%). In a 2^4 factorial design on top of H + B + D + P (16 cells × 500 games at 100 ms) the main effects were +8.7 ± 2.1 points for R (z = 8.1), +5.2 ± 2.2 for K = 0.5 instead of 1.41, +4.1 ± 2.0 for M and +3.6 ± 2.0 for a completion horizon T = 8, with a positive M × K interaction (+3.2 ± 2.2): max-n backups need less exploration. Progressive bias did not help once B was present (−1.2 ± 4.3 points over 2 × 500 games), and letting the opponents in the tree follow the greedy card-value policy cost 16.9 ± 5.8 points.

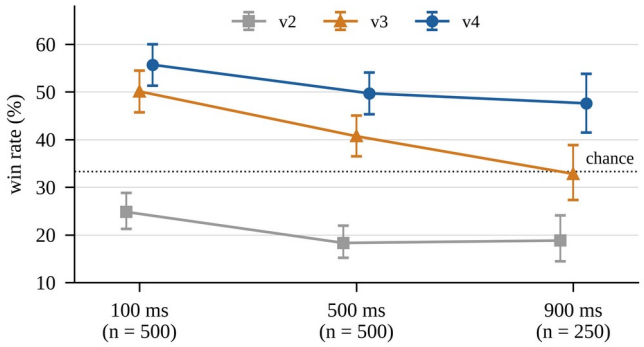


Fig. 2. Win rate with three players against v2 and TAG's ISMCTS by time budget (development runs, seed 1, independent deals). v2 = H + A + B + D, v3 = H + B + D + P + R + M, v4 = v3 + S + I − D. Bars: Wilson 95% intervals.

v3, however, got worse as the budget grew (50.1% → 40.7% → 32.8% at 100, 500 and 900 ms), whereas v4 kept its strength (55.7%, 49.7%, 47.6%), so the gain from S and I grows with the budget (+5.6, +9.0 and +14.8 points), as H2 predicts. Our explanation is that v3's sequential tree let later players react to the cards earlier players had chosen in the same turn; the more iterations, the more its decisions relied on this wrong model, which decoupled UCT removes.

B. Final Agent

TABLE II. Final agent on unseen deals (seed 7)

Setting	Opponents	n	Win % [95% CI]	Margin
3 p, 100 ms	v2, ISMCTS	500	55.0 [50.6, 59.3]	+1.00 ± 0.44
3 p, 500 ms	v2, ISMCTS	500	49.4 [45.0, 53.8]	+0.40 ± 0.39
3 p, 900 ms	v2, ISMCTS	250	56.0 [49.8, 62.0]	+1.06 ± 0.62
4 p, 100 ms	v2, ISMCTS	500	46.8 [42.5, 51.2]	−0.28 ± 0.46
4 p, 500 ms	v2, ISMCTS	500	41.4 [37.2, 45.8]	−1.53 ± 0.46
3 p, 100 ms	ISMCTS × 2 [a]	300	54.2 [48.5, 59.7]	+1.66 ± 0.69
3 p, 100 ms	BasicMCTS, RHEA	500	84.2 [80.7, 87.1]	+6.68 ± 0.60
3 p, 100 ms	OSLA, BasicMCTS	500	87.2 [84.0, 89.8]	+9.74 ± 0.75

Chance: 33.3% (3 players), 25% (4 players). Margin: our score minus the best opponent's, ± 95% CI. [a] run on a 4-core machine (Section IV-E).

Table II reports the final agent on deals not used during development. Against v2 and TAG's ISMCTS it wins 49–56% of the games with three players (chance 33.3%) and 41–

47% with four (chance 25%); against two TAG ISMCTS agents it still wins 54.2%, and against BasicMCTS, RHEA and OSLA 84–87%. With four players its mean margin to the best opponent is slightly negative although it wins far more often than chance: it trades margin for first places.

C. Ablation

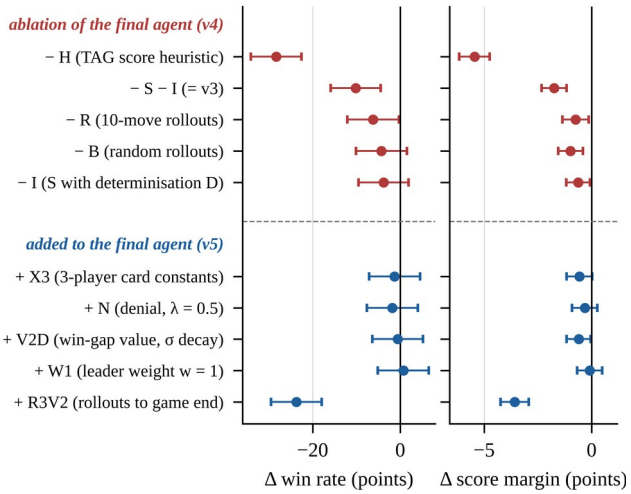


Fig. 3. Paired effects on the final agent with three players at 100 ms (seed 7, 500 games each), ± 95% CI. Top: one component removed. Bottom: additions tested in our last study (Section IV-E).

Fig. 3 (top) removes one component at a time from the final agent. Every removal lowers both measures, and on the score margin every difference is significant ($|z| \geq 2.2$), supporting H1. The heuristic matters most (−28.3 ± 5.8 points without it): TAG's score heuristic ignores unfinished sets and the Maki and Pudding majorities that (1) values. S and I together are worth 10.2 ± 5.7 points ($z = 3.5$) and R 6.2 ± 5.9. The components also change the style of play: at 500 ms (500 games each) v3 scored fewer points per game than v2 from Sashimi (−2.7) and Maki (−1.9) but more from Dumplings (+1.9) and Pudding (+1.3), sets that cannot fail or are decided over the whole game.

D. Four Players and the Limits of Search

With four players the 3-player agent is weaker (46.4% at 100 ms, 38.4% at 500 ms; chance 25%); X0 raises it to 49.6% at 100 ms and X0 + W2 to 45.6% at 500 ms (250 games), and F + L gives 43.8% on other deals. These settings do not transfer back: with three players at 500 ms, adding X0 + W2 to F + L costs 7.7 ± 5.4 points ($z = 2.8$), supporting H3. More search helps little. At four players and 500 ms the tree has about 12 000 iterations but 700 joint actions at the root, 45% of the depth-1 nodes are visited once and the mean depth is 1.6; two searches with different seeds choose the same card in 93–96% of 74 test positions, doubling the time or the rollouts per leaf changes as few decisions, and giving only our agent 1000 ms did not raise its win rate (−4.8 ± 8.1). Yet cutting the tree to depth 1 costs 17.4 ± 8.2 points ($z = 4.2$): the opponents' answer in the next turn matters, but beyond it the evaluation, not the search, limits the agent.

E. What Did Not Help

Our last study (Fig. 3, bottom; seed 7, 500 paired games each on a 4-core machine, where the final agent won 58.1%) tested five additions. Card-model constants tuned for three players with the same greedy tuner (X3) make a far stronger greedy policy – 45.0% against two X0 players, chance 33.3% – but did not strengthen the search (−1.3 ± 5.8 points). Denial ("hate drafting"), in which simulated players also value taking the next player's best card, gained one point in greedy play but nothing in the search (−1.8 ± 5.8). A value aimed at the win gap (V2D) or only at the leader (W1) left the win rate unchanged, and rollouts to the end of the game valued by the win gap cost 23.7 ± 5.8 points ($z = 8.0$). The rollout policy thus has to give unbiased leaf values, not to play well [8], and rollouts must end with the round. In all timing measurements the CPU time per decision stayed below 0.91 s with the 1 s competition file (budget 900 ms, search also stopped after 925 ms of real time).

V. Conclusions and Future Work

We adapted TAG's BasicMCTS to the structure of Sushi Go!: a card-value model for evaluation and simulation, rollouts that end with the round, max-n backups, decoupled UCT for simultaneous moves and information-set search over hidden hands. In paired ablations every component contributes and the heuristic most; matching the tree to simultaneous moves is what lets the agent use longer budgets, and four players need different settings. The final agent wins 55% of three-player games against TAG's tuned ISMCTS and our previous agent, and over 84% against BasicMCTS, RHEA and OSLA.

The study has limits. All opponents are framework agents or our own, so the agent's strength against other designs is unknown; runs at 900 ms and some 4-player runs have 250 games (about ±6 points); and our last study reused the validation deals. The negative results show where progress must come from: the search is close to converged, and a stronger rollout policy, denial or a win-focused value do not help, so the model of the game is now the bottleneck. Future work should learn the evaluation from game outcomes, for example by regressing final ranks on features of round-end states; tune the model's constants by the results of the search rather than by greedy strength; and test regret-matching selection, which suits simultaneous moves better than UCB [12].

Use of Generative AI

Claude (Anthropic), used through the Claude Code tool, was used throughout this project: to write and modify the Java code of our agent on top of BasicMCTS, to write and run the experiment and statistics scripts, and to draft the text, tables and figures of this report from the logs of our runs (repository folders experiments_v3 to experiments_v5, from which every number is taken). The group reviewed the code and the report and takes full responsibility for both. Prompt used to generate this report (original Turkish, then English translation): *"simdi reponun icindeki butun herseyi incele ve hocaya direk yuklyecegimiz raporuda olusutr repoda ornekler tipler brief hersey var 100 alcagimiz bir rapor olsun her raporu tamaladiginda baska bir agent spawn edip bagimsiz durusstce kontrol taki yuzde yuz alacagana emin olankadar looptan cikma"* – "Now examine everything in the repository and also create the report we will upload directly to the lecturer; the repository has examples, templates, the brief, everything. It should be a report that gets 100. Every time you finish the report, spawn another agent to check it independently and impartially, and do not leave the loop until you are sure it will get 100%." The experiments of Section IV-E were run with the prompt "The lecturer will hold a tournament of our agents in class; as before, improve the agents' performance quickly while following the lecturer's restrictions and recording what we did in the README; you have 4.5 hours, test everything in parallel."

References

[1] R. D. Gaina, M. Balla, A. Dockhorn, R. Montoliu, and D. Perez-Liebana, "TAG: A tabletop games framework," in *Proc. AIIDE Workshop on Experimental AI in Games (EXAG)*, 2020.

[2] C.-M. E. Klang, V. Enhörning, A. Alvarez, and J. Font, "Assessing simultaneous action selection and complete information in TAG with Sushi Go!," in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–4.

[3] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.

[4] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. 17th Eur. Conf. Mach. Learn. (ECML)*, 2006, pp. 282–293.

[5] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Mach. Learn.*, vol. 47, no. 2–3, pp. 235–256, 2002.

[6] S. Gelly and D. Silver, “Combining online and offline knowledge in UCT,” in *Proc. 24th Int. Conf. Mach. Learn. (ICML)*, 2007, pp. 273–280.

[7] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, and B. Bouzy, “Progressive strategies for Monte-Carlo tree search,” *New Math. Natural Comput.*, vol. 4, no. 3, pp. 343–357, 2008.

[8] D. Silver and G. Tesauro, “Monte-Carlo simulation balancing,” in *Proc. 26th Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 945–952.

[9] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, “Understanding the success of perfect information Monte Carlo sampling in game tree search,” in *Proc. 24th AAAI Conf. Artif. Intell.*, 2010, pp. 134–140.

[10] I. Frank and D. Basin, “Search in games with incomplete information: A case study using Bridge card play,” *Artif. Intell.*, vol. 100, no. 1–2, pp. 87–123, 1998.

[11] P. I. Cowling, E. J. Powley, and D. Whitehouse, “Information set Monte Carlo tree search,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 120–143, 2012.

[12] M. Lanctot, V. Lisý, and M. H. M. Winands, “Monte Carlo tree search in simultaneous move games with applications to Goofspiel,” in *Computer Games (CGW 2013)*, Commun. Comput. Inf. Sci., vol. 408. Cham, Switzerland: Springer, 2014, pp. 28–43.

[13] M. J. W. Tak, M. Lanctot, and M. H. M. Winands, “Monte Carlo tree search variants for simultaneous move games,” in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2014, pp. 1–8.

[14] N. R. Sturtevant, “An analysis of UCT in multi-player games,” in *Computers and Games (CG 2008)*, Lect. Notes Comput. Sci., vol. 5131. Berlin, Germany: Springer, 2008, pp. 37–49.

[15] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.